INFORMATION SECURITY LAB

MINI PROJECT FINAL REPORT

RowGuard: Machine Learning Based Detection of Suspected Rowhammer Activity

Academic year: [Add academic year]
Prepared by: [Student name(s)] | Roll no.: [Add roll number(s)]
B.Tech, 5th Semester | Section: [Add section]
Course code: [Add course code] | Faculty / guide: [Add name]
Institution / department: [Add affiliation]
4 November 2026

RowGuard: Machine Learning Based Detection of Suspected Rowhammer Activity

[Student name(s)] · [Institution / department] · [Email address(es)]

Abstract

Rowhammer can disturb adjacent DRAM rows when selected rows are activated repeatedly, yet a software monitor cannot directly observe every DRAM activation on commodity machines. This project implements RowGuard, a Linux daemon that samples system-wide hardware performance counters, extracts one-second temporal features, scores them with a trained logistic-regression model, and reports suspected activity in a web dashboard. A three-of-five window policy converts scores into incidents. Incident records are written to a hash-chained, signed local audit log. The dashboard can be viewed locally or through a single-origin Cloudflare Tunnel. Training used 5,703 labelled windows from ten separately timed captures on one Intel Core i5-11400H host; positive labels mean that Google's rowhammer-test was active, not that memory corruption occurred. In run-disjoint grouped evaluation, the deployed-style logistic method achieved 100.0% window precision, 92.7% recall, and zero benign incident openings while detecting all ten test runs. The exact packaged artifact detected both later prospective test runs, with 69.2% window recall and no measured benign incidents. No bit flip was reported. These results establish a reproducible laboratory demonstration on one host and one tool; they do not establish general attack detection or resistance to all benign workloads.

Keywords—Rowhammer; hardware performance counters; machine learning; anomaly detection; audit integrity; web monitoring.

1. Introduction

Rowhammer is a DRAM disturbance phenomenon: repeated activation of aggressor rows can alter data in neighboring rows. Kim et al. demonstrated disturbance errors in commodity memory [1]. Attack patterns have since become more diverse; Blacksmith shows that non-uniform access schedules can bypass defenses that expect regular hammering [2]. A monitor intended for a commodity Linux host therefore needs to distinguish suspicious memory behavior from ordinary high-throughput work, while admitting that performance counters are an indirect signal.

The project objective is a continuously running detector whose entire user-facing experience is one daemon and one monitoring dashboard. Training, attack testing, cryptographic verification, and checkpoint delivery remain internal or maintenance activities. The dashboard must make attack state, feature history, health, and integrity status understandable during a live lab demonstration. An alert is a hypothesis about observed activity; the application does not claim to prevent a physical bit flip.

The research question is practical: with only generic system-wide PMU events on this host, can a small supervised model detect a known hammering test without alerting during the measured benign controls? This report describes the measurement pipeline, the run-disjoint model comparison, the live demonstration, and the boundary between what the data shows and what remains untested.

2. Related Work

Prior work established both the physical disturbance mechanism [1] and the danger of relying on one regular access pattern [2]. Research on performance-counter-based attack detection has argued that microarchitectural event deviations can support machine-learning classifiers for hardware-level attacks [3]. Joardar, Bletsch, and Chakrabarty instead study an ML mitigation design driven by memory access and row information and evaluate a hardware-oriented implementation [4]. RowGuard addresses a narrower software-monitoring setting: it uses event rates available through Linux perf_event_open [5] and produces alerts rather than changing DRAM refresh behavior.

The Duke dataset associated with [4] was considered, but its simulated memory-access/row information has not been shown equivalent to the generic cache, cycle, and instruction counters measured by this daemon [6]. Renaming simulated fields would create a false training/runtime match. The deployable model therefore uses locally captured PMU traces. The Duke data remains a possible separate research benchmark if a compatible feature mapping is

established. Google's rowhammer-test [7] supplies the bounded laboratory workload; its active process label is independent of RowGuard's scores.

3. Methodology

3.1 System architecture

The daemon starts a Linux perf_event_open collector for cycles, instructions, cache references, and cache misses. It samples approximately every 100 ms, checks counter quality, and builds overlapping one-second windows. A feature builder computes cache misses per second, cache-miss ratio, cache-miss coefficient of variation, and instructions per cycle (IPC). The packaged model standardizes the four inputs using training-set parameters and calculates a logistic score. Invalid required measurements do not become zero-valued features. A three-of-five score policy opens an incident; five valid quiet results close it. The dashboard receives status, incidents, feature history, and evidence through same-origin HTTP endpoints; server-sent events provide low-latency updates where supported, with two-second polling as a transport fallback.

Stage	Implemented function	Visible outcome
Collection	System-wide Linux PMU sample; quality flags	MONITORING or explicit DEGRADED state
Detection	Four features → logistic score → 3-of-5 vote	SUSPICIOUS badge and incident timeline
Evidence	Hash chain, Ed25519-signed local checkpoint	Local integrity and anchoring status
Delivery	Same-origin web UI/API; optional tunnel	Dashboard on host or remote browser

Table I. RowGuard processing path and corresponding dashboard output.

3.2 Data and preprocessing

The original corpus contains 5,703 overlapping one-second windows from ten separately timed runs on an Intel Core i5-11400H (12 logical CPUs). Of these, 3,206 windows are labelled benign and 2,497 are labelled tester-active. A lab harness recorded phase boundaries independently of model output. The benign phases include ordinary baseline and a memory-copy control; ambiguous startup and post-test windows were handled explicitly in the corpus builder. Each capture run is kept intact when fitting and testing to avoid placing heavily overlapping windows from the same run on both sides of a split. Standardization is fitted only on training runs. Labels, process names, and run IDs are excluded from predictors.

3.3 Model selection and evaluation

The comparison includes deployed-style logistic regression, a scikit-learn logistic variant, random forest, XGBoost, radial-basis-function support vector machine (RBF SVM), and a fixed PMU rule. Five outer grouped folds each hold out two complete runs. Within the other eight runs, six fit the model and two select a threshold. Threshold choice first minimizes benign incident openings, then maximizes detected attack runs, then window F1. Every candidate uses the same three-of-five incident policy. This follows grouped-split practice [8], though all folds still originate from one host and one tool. The deployed-style row retrains the same equations as the packaged detector in each fold; a separate later check evaluates the exact packaged artifact.

Window precision, recall, accuracy, and F1 are reported alongside incident openings, because an operator experiences incident alerts rather than isolated window decisions. Ten-run grouped results are comparative estimates, not confidence bounds for other machines. The test sessions are short and the benign corpus represents about five minutes of observation, which is too little to estimate a long-term false-alert rate reliably.

3.4 Evidence and remote access

Audit records include a sequence and previous-record hash, and local checkpoints are signed with Ed25519. Verification detects edits to the signed local chain. A separate receiver can acknowledge checkpoints, but no independent off-host receiver was configured for these lab runs; local signatures alone cannot exclude whole-history

rollback by a host administrator. “Clear visible log” changes only dashboard visibility and appends an audit marker. For a remote demonstration, cloudflared forwards the whole dashboard and API under one HTTPS hostname while RowGuard remains on 127.0.0.1. Cloudflare documents that Quick Tunnels do not support SSE [9], so the browser polls the same-origin API every two seconds in that mode. Access can be restricted by email for the temporary URL; a named tunnel plus Access is preferable for a stable deployment [10].

4. Results and Discussion

4.1 Grouped model comparison

Table II reports the evaluation of the model method on all ten held-out runs. “Benign incidents” counts incident openings during benign phases after the same vote policy, not false-positive windows. The fixed rule confirms that incident-level success alone can hide poor window recall.

Model	Precision	Recall	Accuracy	F1	Runs	Benign alerts
Deployed-style logistic	100.0%	92.7%	96.8%	96.2%	10/10	0
scikit-learn logistic	99.4%	95.7%	97.8%	97.5%	10/10	3
Random forest	99.3%	92.1%	96.2%	95.5%	10/10	3
XGBoost	99.6%	89.8%	95.4%	94.5%	10/10	1
RBF SVM	99.1%	89.3%	95.0%	94.0%	10/10	2
Fixed PMU rule	100.0%	24.7%	67.1%	39.7%	10/10	0

Table II. Run-disjoint grouped comparison on 5,703 one-second windows. Precision, recall, accuracy, and F1 are window-level; alert counts are incident-level.

The packaged logistic model was selected because the project’s principal demonstration requirement was to alert during the Google test without generating visible incidents in measured ordinary activity. Its method matched the best benign-incident count while retaining much higher window recall than the fixed rule. The scikit-learn logistic variant improved accuracy, F1, and recall, but opened three benign incidents. XGBoost and RBF SVM did not dominate the chosen method on that combined user-facing objective. Logistic inference is also simple to package and reproduce inside the Python daemon. Selection is therefore specific to the present host, tool, controls, and threshold policy; it is not a claim that logistic regression is universally best for Rowhammer detection.

4.2 Fresh runs and live demonstration

Two later prospective runs were excluded from the ten-run corpus. The exact packaged logistic artifact reached 100.0% precision and 69.2% recall over 1,039 evaluated windows, detected both tester-active runs, and opened no incidents during the measured baseline or memory-copy controls. The RBF SVM reached 87.9% recall on these same runs with zero benign incidents, making it a promising candidate for a larger follow-up study. The variation from 92.7% grouped method recall to 69.2% prospective artifact recall underscores the fragility of a small, related dataset. One live incident appeared roughly 0.6 seconds after the tester stopped because a rolling feature window still contained attack-period samples; this visible timing artifact should not be read as a clean post-test false alert.

In the fresh live check, the daemon reported the packaged logistic detector, opened an incident during a bounded 35-second run of Google’s tool, and opened none in the measured benign phases. The dashboard showed the SUSPICIOUS state and incident history. The top CPU process was displayed as an unverified candidate; system-wide PMU counters cannot prove per-process causation. The tool reported no bit flips. Therefore, the experiment demonstrates detection of that tool’s activity, not successful physical memory corruption.

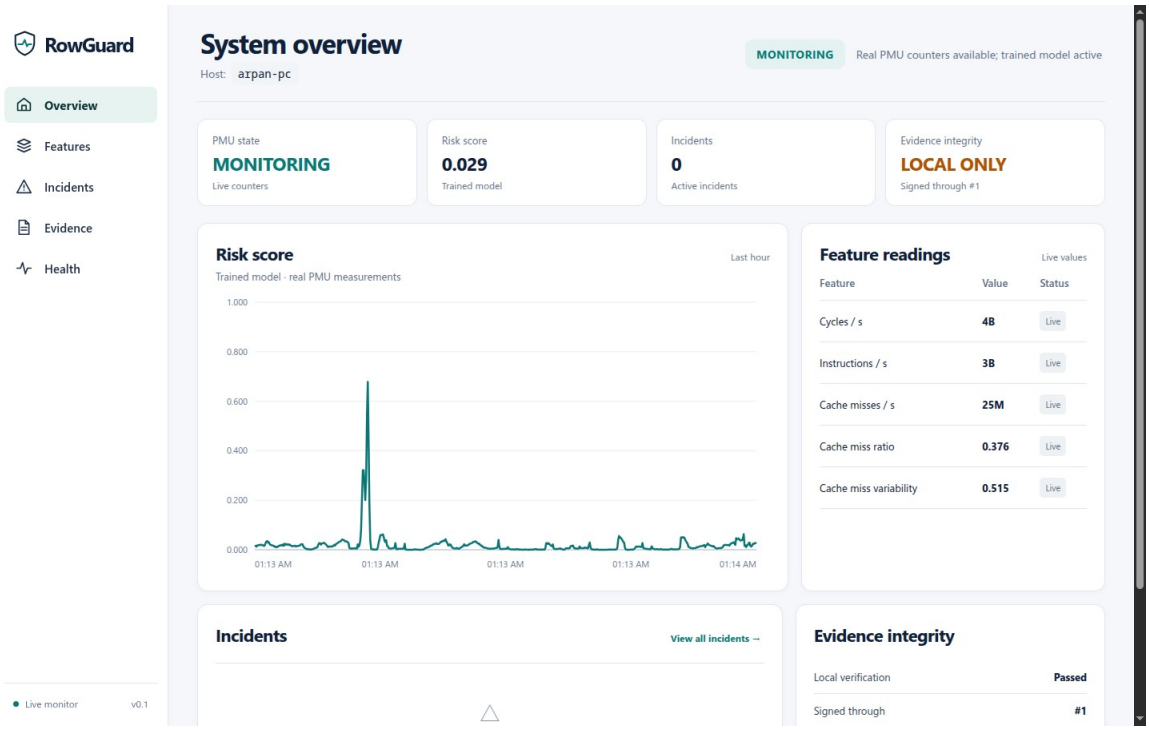


Figure 1. Live desktop dashboard on the measured host. LOCAL ONLY correctly indicates that no independent off-host checkpoint was configured for this capture.

4.3 Integrity and threats to validity

Local signature and hash-chain verification passed on the recorded lab captures. A deliberate record alteration and signed-tail truncation are rejected in automated tests. However, the lab has no independent off-host checkpoint; a compromised host with control of logs and local trust material remains outside the demonstrated integrity guarantee. The system's "tamper-evident" terminology is intentional. The remote tunnel is a viewing path and does not change this trust boundary.

The main validity threats are one CPU/DRAM configuration, one test program, short benign observation, highly correlated rolling windows, and limited workload diversity. No Blacksmith or other non-uniform hammering family was tested live. Generic cache misses are a proxy, not a count of DRAM row activations. The sigmoid score is not calibrated as a physical attack probability. Zero observed benign incidents across a few minutes cannot support an "only Rowhammer alerts" guarantee. Longer benign soaks, multiple boots, held-out hosts, different DIMMs and attack families, and independent bit-flip evidence are necessary before generalization.

5. Conclusion and Future Work

RowGuard provides an end-to-end laboratory prototype: a Linux PMU daemon, an ML-based alert policy, a local/tunnel-accessible monitoring dashboard, and cryptographically verifiable local evidence. The current model choice is supported by run-disjoint comparison and a fresh live demonstration, with the explicit trade-off that its prospective window recall is lower than an RBF SVM candidate. The project's strongest claim is narrow: on the tested host, bounded Google rowhammer-test runs produced visible incidents while the measured short benign controls did not. No bit flip was observed.

Next work should collect hours of diverse benign traffic and independent attack families, compare incident delay and alert burden across machines, improve incident grouping around short score gaps, and validate feature transfer between CPU profiles. A separately administered checkpoint receiver should be deployed before claiming rollback detection. The dashboard and detector should continue to expose missing measurements and uncertainty rather than implying that a clean score proves a clean machine.

References

[1] Y. Kim et al., “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors,” Proc. 41st Int. Symp. Computer Architecture, 2014. https://www.cs.cmu.edu/~18742/papers/Kim2014.pdf

[2] P. Jattke, V. van der Veen, P. Frigo, S. Gunter, and K. Razavi, “BLACKSMITH: Scalable Rowhammering in the Frequency Domain,” IEEE Symp. Security and Privacy, pp. 716–734, 2022, doi: 10.1109/SP46214.2022.9833772.

[3] C. Li and J.-L. Gaudiot, “Detecting Malicious Attacks Exploiting Hardware Vulnerabilities Using Performance Counters,” IEEE COMPSAC, pp. 588–597, 2019, doi: 10.1109/COMPSAC.2019.00090.

[4] B. K. Joardar, T. K. Bletsch, and K. Chakrabarty, “Machine Learning-Based Rowhammer Mitigation,” IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems, vol. 42, no. 5, 2023, doi: 10.1109/TCAD.2022.3206729.

[5] Linux man-pages project, “perf_event_open(2).” https://man7.org/linux/man-pages/man2/perf_event_open.2.html (accessed Nov. 4, 2026).

[6] Duke Research Data Repository, “Data from: Machine Learning-based Rowhammer Mitigation,” version 1, doi: 10.7924/r4hh6p604. https://research.repository.duke.edu/record/130

[7] Google, “rowhammer-test,” GitHub repository. https://github.com/google/rowhammer-test (accessed Nov. 4, 2026).

[8] scikit-learn developers, “GroupKFold.” https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html (accessed Nov. 4, 2026).

[9] Cloudflare, “Quick Tunnels.” https://developers.cloudflare.com/tunnel/get-started/quick-tunnels/ (accessed Nov. 4, 2026).

[10] Cloudflare, “Set up Cloudflare Tunnel” and “Private web application.” https://developers.cloudflare.com/tunnel/get-started/ ; https://developers.cloudflare.com/cloudflare-one/setup/secure-private-apps/private-web-app/ (accessed Nov. 4, 2026).

Project evidence: docs/MODEL_COMPARISON.md; docs/MODEL_COMPARISON_PROSPECTIVE.json; docs/ML_EVALUATION.md; docs/HARDWARE_EVALUATION.md; ml/data/rowguard_pmu_training.manifest.json. Repository snapshot dated 4 November 2026.